

RAG

دليل عملي للبحث والاسترجاع في أنظمة

Semantic Search - Search Techniques - Hybrid Search - Retrieval Evaluation

Python Code + LangChain + Chroma + BM25 + شرح عربي

هدف الدليل: تحويل المفاهيم النظرية إلى Pipeline عملية قابلة للتجربة، مع توضيح لماذا نستخدم كل تقنية ومتى تكون مناسبة.

إعداد تعليمي من محتوى المحادثة والترانسكربتات المرفقة

خريطة الدليل

1. بنية الـ Retriever داخل RAG

2. Metadata Filtering

3. BM25 و Keyword Search: TF-IDF

4. Semantic Search والـ Embeddings

5. مقاييس التشابه: Euclidean و Cosine و Dot Product

6. Chroma والفرق بين Vector Store و Retriever

7. Hybrid Search وخطوات الدمج

8. Reciprocal Rank Fusion والأوزان

9. تقييم الـ Retriever: Precision و Recall و MAP و MRR

10. مشروع Python متكامل وخطة Tuning

ملاحظة المصطلحات: سيظل الكود والمصطلحات التقنية الأساسية بالإنجليزية، بينما الشرح باللغة العربية لتسهيل الربط بين الدراسة والتطبيق.

1. الـ Retriever داخل نظام RAG

الـ Retriever هو المكوّن المسؤول عن استقبال سؤال المستخدم وإرجاع أكثر المستندات أو الـ Chunks ارتباطًا به. جودة الإجابة النهائية تعتمد بقوة على جودة المستندات التي يعيدها؛ لأن الـ LLM لا يستطيع الاستفادة من معلومة لم تصل إلى الـ Context.

RAG Retrieval Flow



Vector Store vs Retriever

العنصر	المسؤولية	مثال
Vector Store	تخزين الـ Embeddings والمستندات وتنفيذ البحث المتجهي	Chroma, FAISS, Pinecone
Retriever	واجهة موحدة تستقبل Query وتعيد Documents	<code>()vector_store.as_retriever</code>

```

# Direct use of the vector store
results = vector_store.similarity_search(query, k=3)

# Standard retriever interface
retriever = vector_store.as_retriever(search_kwargs={"k": 3})
results = retriever.invoke(query)
  
```

الفكرة: `as_retriever()` لا ينسخ البيانات ولا ينشئ قاعدة جديدة؛ بل يلفّ الـ `Vector Store` داخل واجهة قياسية يمكن توصيلها بباقي `LangChain`.

2. Metadata Filtering

الـ `Metadata Filtering` يستخدم شروطًا صارمة مخزنة مع كل مستند. النتيجة هنا `Yes/No`: المستند إما يطابق الشرط أو لا يطابقه. وهو سريع وسهل التفسير، لكنه لا يفهم معنى السؤال وحده.

```
from langchain_core.documents import Document

Document(
    page_content="LangChain builds LLM applications.",
    metadata={
        "topic": "AI",
        "department": "engineering",
        "year": 2026,
    },
)
```

Chroma metadata filter

```
retriever = vector_store.as_retriever(
    search_kwargs={
        "k": 5,
        "filter": {"topic": "AI"},
    }
)

docs = retriever.invoke("How do I build an LLM application?")
```

استخدمه عندما يكون عندك قيود واضحة مثل الشركة، القسم، نوع الملف، اللغة، التاريخ، الصلاحيات أو العميل. لا تعتمد عليه وحده للبحث الدلالي.

Keyword Search .3

البحث بالكلمات يرتّب المستندات بناءً على وجود الكلمات نفسها في السؤال والمستند. يكون قويًا جدًا مع أسماء المنتجات، Error Codes، أسماء Classes، أرقام العقود والمصطلحات التقنية الدقيقة.

TF-IDF باختصار

TF يقيس تكرار الكلمة داخل المستند، وIDF يقلل قيمة الكلمات الشائعة عبر كل المستندات. لذلك تحصل الكلمات المميزة للمستند على وزن أعلى.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = [
    "LangChain builds LLM applications",
    "Python is used in machine learning",
    "Pizza is an Italian food",
]

vectorizer = TfidfVectorizer()
document_matrix = vectorizer.fit_transform(corpus)
query_vector = vectorizer.transform(["LangChain applications"])

scores = cosine_similarity(query_vector, document_matrix).flatten()
ranking = scores.argsort()[::-1]

for index in ranking:
    print(index, scores[index], corpus[index])
```

BM25

BM25 تطوير عملي للبحث بالكلمات، ويوازن بين تكرار الكلمة وطول المستند ومدى ندرة المصطلح. غالبًا يكون أفضل من TF-IDF في محركات الاسترجاع النصي.

```
from langchain_community.retrievers import BM25Retriever
```

```
bm25_retriever = BM25Retriever.from_documents(documents)
bm25_retriever.k = 5

results = bm25_retriever.invoke("LangChain retriever")
```

نقطة قوة: Keyword Search يحافظ على Exact Match. لذلك لا تستبدله دائمًا بالـ Semantic Search، خصوصًا مع الأكواد والأسماء.

Semantic Search .4

الـ Semantic Search يبحث بالمعنى بدل التطابق الحرفي. يتم تحويل المستند والسؤال إلى Vectors باستخدام Embedding Model، ثم تتم مقارنة موقع واتجاه الـ Vectors للعثور على أقرب المعاني.

```
Documents -> Embedding Model -> Document Vectors -> Vector Store
Query      -> Embedding Model -> Query Vector      -> Similarity Search
```

كيف تتعلم Embedding Models؟

أثناء التدريب تُستخدم أزواج متشابهة Positive Pairs وأزواج غير مرتبطة Negative Pairs. النموذج يتعلم تقريب تمثيلات الجمل المتشابهة وإبعاد غير المتشابهة، وغالبًا يسمى هذا Contrastive Learning.

قاعدة مهمة: لا تقارن Vectors خرجت من Embedding Models مختلفة؛ كل Model ينشئ فضاء Vector Space خاصًا به.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
```

```

query_vector = embeddings.embed_query(
    "How can I build AI applications?"
)

document_vectors = embeddings.embed_documents([
    "LangChain is used to build LLM applications.",
    "Lions live in Africa.",
])

print(len(query_vector))

```

5. مقاييس التشابه والمسافة

Dot Product

الـ Dot Product هو مجموع ضرب العناصر المناظرة. الناتج Scalar واحد. هندسيًا يعبر عن مقدار توافق الاتجاه، لكنه يتأثر أيضًا بطول الـ Vectorين.

```

import numpy as np

vector_a = np.array([2.0, 0.0])
vector_b = np.array([3.0, 4.0])

dot_product = np.dot(vector_a, vector_b)
print(dot_product) # 6.0

```

العلاقة الهندسية: $A \cdot B = ||A|| \times ||B|| \times \cos(\theta)$. لو الاتجاهان متشابهان تكون القيمة موجبة وكبيرة، ولو متعامدين تكون صفرًا، ولو متعاكسين تكون سالبة.

Projection ليست هي Dot Product

```

norm_a = np.linalg.norm(vector_a)

```

```
projection_length = dot_product / norm_a
projection_vector = (
    dot_product / (norm_a ** 2)
) * vector_a

print(projection_length) # 3.0
print(projection_vector) # [3. 0.]
```

الفرق: نحن نحسب Dot Product كعملية، بينما Projection تفسير هندسي مرتبط به. Dot Product = طول Vector المرجع × طول الإسقاط عليه.

Cosine Similarity

يقيس تشابه الاتجاه فقط بعد إزالة تأثير الطول. قيمته عادة بين -1 و1: نفس الاتجاه قريب من 1، التعامد 0، الاتجاه المعاكس قريب من -1.

```
def cosine_similarity(a, b):
    numerator = np.dot(a, b)
    denominator = np.linalg.norm(a) * np.linalg.norm(b)
    if denominator == 0:
        return 0.0
    return numerator / denominator

print(cosine_similarity(vector_a, vector_b))
```

Euclidean Distance

تقيس المسافة المستقيمة بين نقطتين. كلما قلت المسافة زاد التشابه. لكنها قد تصبح أقل تمييزاً في الأبعاد العالية. مقارنة ببعض البدائل.

```
distance = np.linalg.norm(vector_a - vector_b)
print(distance)
```


المقياس	ما الذي يقيسه؟	الأفضل عندما
Dot Product	الاتجاه + الطول	الـ Embeddings normalized أو عندما يحمل الطول معنى
Cosine Similarity	الاتجاه فقط	تشابه المعنى مع تجاهل magnitude
Euclidean Distance	المسافة الهندسية	الفضاء والتدريب متوافقان مع L2

6. Semantic Search باستخدام Chroma

```
# pip install langchain langchain-openai langchain-chroma python-dotenv
```

```
from dotenv import load_dotenv
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

load_dotenv()

documents = [
    Document(
        page_content="LangChain builds LLM applications.",
        metadata={"document_id": "doc_1", "topic": "AI"},
    ),
    Document(
        page_content="RAG combines retrieval and generation.",
        metadata={"document_id": "doc_2", "topic": "AI"},
    ),
    Document(
        page_content="Pizza is a popular Italian dish.",
        metadata={"document_id": "doc_3", "topic": "Food"},
    ),
]

embedding_model = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

vector_store = Chroma.from_documents(
    documents=documents,
```

```

embedding=embedding_model,
collection_name="rag_knowledge_base",
persist_directory="./chroma_db",
)

retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3},
)

results = retriever.invoke(
    "How do retrieval-based AI applications work?"
)

for rank, doc in enumerate(results, start=1):
    print(rank, doc.metadata, doc.page_content)

```

Consistency: استخدم نفس Embedding Model عند Indexing المستندات وعند Embedding السؤال.

Hybrid Search .7

Hybrid Search يجمع نقاط القوة: Keyword Search للتطابق الدقيق، Semantic Search للتشابه في المعنى، وMetadata Filtering للقيود الصارمة. بعد ذلك تُدمج القوائم المرتبة في Ranking نهائي.

```

Query
|-- BM25 Keyword Search -----> Ranked List A
|-- Vector Semantic Search -----> Ranked List B

Ranked Lists -> Metadata Filter -> Rank Fusion -> Final Top-K

```

خطوات الـ Pipeline

1. استقبال سؤال المستخدم.

2. تشغيل BM25 و Semantic Search بالتوازي.
3. الحصول على قائمتين Ranked Lists.
4. تطبيق Metadata Filters لإزالة النتائج غير المسموح بها أو غير المناسبة.
5. دمج القائمتين باستخدام Reciprocal Rank Fusion.
6. إرجاع أفضل Top-K Documents إلى LLM.

8. Reciprocal Rank Fusion (RRF)

RRF يعتمد على ترتيب المستند داخل كل قائمة، وليس على الـ Raw Score الناتج من BM25 أو Vector Search. هذا مفيد لأن Scores التقنيتين ليست على نفس المقياس.

Weighted RRF formula

```
RRF_score(document) = sum(
    weight_i / (c + rank_i(document))
)
```

الثابت c أو K يقلل سيطرة المركز الأول. عند قيمة صغيرة يكون الفرق بين Rank 1 و Rank 10 كبيرًا، وعند قيمة مثل 50 أو 60 يصبح الدمج أكثر توازنًا.

```
from collections import defaultdict

def weighted_rrf(ranked_lists, weights, constant=60):
    scores = defaultdict(float)

    for ranked_ids, weight in zip(ranked_lists, weights):
        for rank, document_id in enumerate(ranked_ids, start=1):
            scores[document_id] += weight / (constant + rank)

    return sorted(
        scores.items(),
        key=lambda item: item[1],
        reverse=True,
    )
```

```
keyword_ranking = ["doc_1", "doc_3", "doc_2"]
semantic_ranking = ["doc_2", "doc_1", "doc_4"]

final_ranking = weighted_rrf(
    ranked_lists=[keyword_ranking, semantic_ranking],
    weights=[0.3, 0.7],
    constant=60,
)

print(final_ranking)
```

LangChain Hybrid Retriever

```
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

bm25_retriever = BM25Retriever.from_documents(documents)
bm25_retriever.k = 5

vector_retriever = vector_store.as_retriever(
    search_kwargs={"k": 5}
)

hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.3, 0.7],
    c=60,
)

results = hybrid_retriever.invoke(
    "How does LangChain retrieval work?"
)
```

Tuning Start: 70% Semantic و 30% Keyword نقطة بداية شائعة، وليست قاعدة ثابتة. التطبيقات التي تعتمد على أسماء وأكواد دقيقة قد تحتاج وزنًا أعلى للـ Keyword Search.

9. تقييم جودة الـ Retriever

لتقييم البحث تحتاج: Query، قائمة النتائج المرتبة، و Ground Truth يحدد جميع المستندات الصحيحة. بدون Ground Truth لا يمكن حساب المقاييس الحتمية بدقة.

Precision@K

من بين أول K مستندات تم استرجاعها، ما نسبة المستندات الصحيحة؟ يقيس مدى نظافة وثقة النتائج.

$$\text{Precision@K} = \text{Relevant Retrieved in Top-K} / K$$

Recall@K

من بين جميع المستندات الصحيحة الموجودة في قاعدة المعرفة، ما النسبة التي نجح Top-K في العثور عليها؟ يقيس الشمولية وعدم تفويت المعلومات.

$$\text{Recall@K} = \text{Relevant Retrieved in Top-K} / \text{All Relevant Documents}$$

MAP, Average Precision

Average Precision يكافئ ظهور المستندات الصحيحة مبكرًا. نحسب Precision عند كل Rank يحتوي على مستند صحيح ثم نأخذ المتوسط. MAP هو متوسط AP عبر مجموعة Queries.

MRR, Reciprocal Rank

Reciprocal Rank يهتم بمكان أول مستند صحيح فقط: $1 / \text{rank}$. و MRR هو المتوسط عبر Queries كثيرة. مناسب عندما تكون أول إجابة صحيحة أهم من بقية القائمة.

Metric	السؤال الذي تجيب عنه	متى تهتم؟
Precision@K	كم نتيجة صحيحة داخل Top-K؟	عندما تريد Context نظيفًا

Metric	السؤال الذي تجيب عنه	متى تهم؟
Recall@K	كم معلومة صحيحة تم العثور عليها؟	المقياس الأساسي للـ Retriever
MAP@K	هل النتائج الصحيحة مرتبة عاليًا عمومًا؟	تقييم جودة Ranking كامل
MRR	متى ظهر أول مستند صحيح؟	FAQ و Search حيث أول نتيجة مهمة

10. كود Metrics من الصفر

```

from collections.abc import Sequence

def precision_at_k(retrieved_ids, relevant_ids, k):
    top_k = list(retrieved_ids[:k])
    if not top_k:
        return 0.0

    relevant_count = sum(
        doc_id in relevant_ids for doc_id in top_k
    )
    return relevant_count / len(top_k)

def recall_at_k(retrieved_ids, relevant_ids, k):
    if not relevant_ids:
        return 0.0

    top_k = retrieved_ids[:k]
    relevant_count = sum(
        doc_id in relevant_ids for doc_id in top_k
    )
    return relevant_count / len(relevant_ids)

def average_precision_at_k(retrieved_ids, relevant_ids, k):

```

```

relevant_seen = 0
precision_sum = 0.0

for rank, doc_id in enumerate(retrieved_ids[:k], start=1):
    if doc_id in relevant_ids:
        relevant_seen += 1
        precision_sum += relevant_seen / rank

return precision_sum / relevant_seen if relevant_seen else 0.0

```

```

def reciprocal_rank(retrieved_ids, relevant_ids):
    for rank, doc_id in enumerate(retrieved_ids, start=1):
        if doc_id in relevant_ids:
            return 1.0 / rank
    return 0.0

```

```

retrieved = ["doc_1", "doc_7", "doc_3", "doc_9", "doc_5"]
relevant = {"doc_1", "doc_9", "doc_5", "doc_10"}

print("Precision@5:", precision_at_k(retrieved, relevant, 5))
print("Recall@5:", recall_at_k(retrieved, relevant, 5))
print("AP@5:", average_precision_at_k(retrieved, relevant, 5))
print("RR:", reciprocal_rank(retrieved, relevant))

```

11. Metrics وأدوات جاهزة

لا يلزم كتابة كل شيء يدويًا. لكن يجب التفرقة بين ID-based Retrieval Metrics وبين LLM-as-a-Judge metrics.

الأداة	الاستخدام الأفضل	ملاحظات
Ragas	Context Precision/Recall و Answer Relevancy و Faithfulness	قد يستخدم LLM ويضيف تكلفة وعدم حتمية
LangSmith	Datasets وتجارب ومقارنة Versions و Monitoring	مناسب مع LangChain وال Production

الأداة	الاستخدام الأفضل	ملاحظات
scikit-learn	مقاييس عامة للتصنيف والـ PR curves	ليست كل الدوال مطابقة مباشرة لـ Ranked Retrieval
Custom ID Metrics	Recall@K و MRR حتمية	تحتاج Ground Truth IDs ثابتة

```
# Example installation
# pip install ragas langsmith scikit-learn

# Keep deterministic retrieval metrics based on document IDs,
# and use Ragas/LangSmith for broader RAG evaluation.
```

Evaluating a LangChain Retriever .12

```
def evaluate_retriever(retriever, dataset, k=5):
    precision_scores = []
    recall_scores = []
    ap_scores = []
    rr_scores = []

    for sample in dataset:
        docs = retriever.invoke(sample["query"])
        retrieved_ids = [
            doc.metadata["document_id"]
            for doc in docs[:k]
        ]
        relevant_ids = set(sample["relevant_document_ids"])

        precision_scores.append(
            precision_at_k(retrieved_ids, relevant_ids, k)
        )
        recall_scores.append(
            recall_at_k(retrieved_ids, relevant_ids, k)
        )
        ap_scores.append(
            average_precision_at_k(retrieved_ids, relevant_ids, k)
        )
        rr_scores.append(
            reciprocal_rank(retrieved_ids, relevant_ids)
        )
```



```

count = len(dataset)
return {
    f"precision@{k}": sum(precision_scores) / count,
    f"recall@{k}": sum(recall_scores) / count,
    f"map@{k}": sum(ap_scores) / count,
    "mrr": sum(rr_scores) / count,
}

evaluation_dataset = [
    {
        "query": "What is LangChain?",
        "relevant_document_ids": ["doc_1"],
    },
    {
        "query": "How does RAG work?",
        "relevant_document_ids": ["doc_2"],
    },
]

metrics = evaluate_retriever(
    hybrid_retriever,
    evaluation_dataset,
    k=5,
)

print(metrics)

```

13. خطة Tuning عملية

7. أنشئ Evaluation Dataset تمثل أسئلة المستخدمين الفعلية، وليس أسئلة سهلة فقط.
8. ابدأ بـ BM25 فقط و Semantic فقط للحصول على Baselines منفصلة.
9. جرّب Hybrid Weights مثل 30/70 و 50/50 و 70/30.
10. جرّب Top-K مختلفًا مثل 3 و 5 و 10، وراقب Recall مقابل Precision وتكلفة الـ Context.
11. اختبر Chunk Size و Chunk Overlap؛ جودة الـ Chunks تؤثر على الاسترجاع بقدر اختيار الـ Model.
12. استخدم Metadata Filtering قبل أو أثناء البحث عندما تكون هناك قيود مؤكدة.

13. اختر الإعداد الأفضل بناءً على Metrics وليس الانطباع من سؤال أو سؤالين.

14. راقب Latency والتكلفة مع الجودة؛ أفضل Retriever نظريًا قد لا يكون مناسبًا للإنتاج.

14. مشروع نهائي مختصر

```
# 1) Build documents with stable IDs and metadata
# 2) Create BM25 keyword retriever
# 3) Create Chroma semantic retriever
# 4) Combine them using EnsembleRetriever
# 5) Retrieve Top-K documents
# 6) Evaluate using Recall@K, Precision@K, MAP@K, and MRR
# 7) Tune weights, k, filters, and chunking

query = "How can I build a RAG application with LangChain?"
retrieved_docs = hybrid_retriever.invoke(query)

for rank, doc in enumerate(retrieved_docs, start=1):
    print(
        rank,
        doc.metadata.get("document_id"),
        doc.page_content,
    )
```

الخلاصة

- Metadata Filtering يطبق شروطًا صارمة وسريعة.
- Keyword Search ممتاز للـ Exact Match والمصطلحات التقنية.
- Semantic Search يفهم المعنى باستخدام Embeddings.
- Chroma يخزن وينفذ البحث، والـ Retriever يوفر واجهة موحدة.
- Hybrid Search يجمع القوائم ويوازن نقاط القوة، غالبًا باستخدام RRF.
- Recall@K هو أساس تقييم الاسترجاع، ثم Precision و MAP و MRR حسب هدف التطبيق.
- لا يوجد إعداد مثالي للجميع؛ Evaluation Dataset و Tuning هما الطريق الصحيح.

قاعدة **Production:** Garbage retrieved, garbage generated. تحسين الـ LLM لن يعالج Retriever يعيد

Context غير مناسب.

ملاحظات المصدر

تم إعداد هذا الدليل اعتمادًا على الشرح المتبادل في المحادثة والترانسكربتات التعليمية التي قدمها المستخدم عن Retrieval Evaluation و Hybrid Search و Semantic Search. صيغ الكود هنا تعليمية وعملية، وقد تحتاج تعديلات بسيطة حسب إصدارات المكتبات وبيئة المشروع.